

Curso: Técnicas de Programación

DOCUMENTACIÓN TÉCNICA FINAL

Proyecto #1 – Vitta

Sistema de gestión nutricional desarrollado en C# con Windows Forms, basado en arquitectura MVC y persistencia local en archivos CSV.

Estudiante	Natalia Tobal D.
Profesor	Luis Felipe Mora Umaña.
Fecha de entrega	07 de abril de 2026
Repositorio	https://github.com/Nat-92CR/ProyectoVitta.git
Gestión del proyecto	https://trello.com/b/3z5MMCwt/proyecto-sistema-de-gestion-nutricional-vitta
Versión documentada	Proyecto Vitta - Iteración 1

Índice

1.	Introducción	3
2.	Decisiones de diseño	4
3.	Desarrollo del sistema y explicación de módulos	6
4.	Uso, ejecución e interacciones del sistema.....	9
5.	Análisis de resultados	12
6.	Aprendizajes del desarrollo	13
7.	Conclusiones	14
8.	Bibliografía.....	14

1. Introducción

Vitta es una aplicación de escritorio orientada al registro y control de la alimentación diaria. El proyecto fue desarrollado como primera iteración del curso Técnicas de Programación, por lo que el enfoque principal estuvo en construir una base extensible, comprensible y organizada, capaz de crecer hacia una segunda fase sin tener que rehacer toda la lógica del sistema.

El problema planteado por el proyecto consiste en permitir que un usuario se registre, gestione alimentos, construya menús diarios, consulte información nutricional y visualice estadísticas por fecha. Para responder a ese planteamiento se utilizó C# con Windows Forms en la capa visual, una separación por proyectos siguiendo arquitectura Modelo-Vista-Controlador y una persistencia simple basada en archivos CSV.

Además del funcionamiento, la entrega exige evidencia de buenas prácticas de desarrollo. Por esa razón el proyecto fue estructurado en tres capas, se utilizaron interfaces para desacoplar responsabilidades, se centralizó la configuración de los archivos en ConfigurationItems.cs, se incorporó análisis estático con SonarAnalyzer y se integró un patrón Strategy para el cálculo de macronutrientes según el tipo de dieta.

En las siguientes secciones se presentan las decisiones de diseño adoptadas, el desarrollo general del sistema, la forma de ejecución e interacción, así como el análisis de resultados, los aprendizajes obtenidos y las conclusiones de esta primera iteración.

2. Decisiones de diseño

Durante el desarrollo de Vitta no solo se buscó que el sistema cumpliera con las funciones solicitadas, sino también que tuviera una estructura ordenada, comprensible y fácil de trabajar. Por eso, a lo largo de la implementación se tomaron distintas decisiones de diseño relacionadas con la organización del proyecto, el manejo de los datos, la distribución de responsabilidades y la forma en que el usuario interactúa con el sistema.

Estas decisiones fueron importantes porque permitieron construir una solución más clara, más mantenible y coherente con lo solicitado en el proyecto. Además, ayudan a entender por qué el sistema fue desarrollado de esta manera y qué propósito cumple cada parte dentro del funcionamiento general de Vitta.

- **Separación por proyectos:**

La solución se dividió en VittaModel, VittaController y VittaView con el propósito de mantener responsabilidades claramente diferenciadas. Esta decisión evita mezclar la lógica de negocio, la gestión de datos y la interfaz gráfica dentro de un mismo espacio de trabajo. De esta manera, el modelo concentra las entidades principales del sistema, la capa de control coordina el comportamiento y la vista se limita a la interacción con el usuario. Esta organización hace que el proyecto resulte más comprensible, más ordenado y sencillo de revisar.

- **Uso de interfaces en la capa de control:**

En la capa de control se incorporaron interfaces como IUserController, IFoodController, IMenuController, INutritionInfoController e IStatisticsController. El objetivo de esta decisión fue desacoplar la definición de la responsabilidad de su implementación concreta. En términos prácticos, esto permite que cada controlador cumpla un contrato claro y que, en una futura evolución del sistema, pueda modificarse una implementación sin obligar a rehacer toda la estructura. Además, esta decisión mejora la legibilidad del proyecto, porque deja explícito qué funciones debe cumplir cada módulo.

- **Persistencia local en archivos CSV:**

Debido a que la primera iteración del proyecto debía trabajar con archivos y no con una base de datos, se optó por utilizar archivos CSV como mecanismo de almacenamiento. Esta decisión simplifica la carga, lectura y validación de la información, y además mantiene la solución alineada con lo solicitado en el enunciado. El uso de CSV también permite revisar fácilmente el contenido de los registros desde fuera del sistema, algo útil tanto para pruebas como para validación académica de los datos utilizados en la entrega.

- **Manejador genérico de persistencia:**

Para no repetir la misma lógica de lectura y escritura en diferentes módulos, se implementó la clase `FileHandler<T>`. Esta decisión permite reutilizar una estructura común para distintos tipos de datos, manteniendo una solución más uniforme y reduciendo código repetido. En lugar de crear una clase independiente para cada archivo, el sistema aprovecha un mismo manejador adaptable a diferentes modelos, lo cual contribuye a una implementación más limpia y mantenible.

- **Centralización de la configuración:**

Las rutas de los archivos de usuarios, alimentos y menús se agruparon en `ConfigurationItems.cs`. En la versión final, además, los CSV se organizaron en la carpeta `data` y la aplicación trabaja con rutas relativas basadas en el directorio de ejecución. Esta decisión evita depender de rutas absolutas de una sola computadora, mantiene la configuración en un punto único y facilita mover el proyecto sin romper la persistencia.

- **Uso del patrón Strategy en la información nutricional:**

Para fortalecer la separación de responsabilidades en el cálculo nutricional, se implementó el patrón Strategy mediante la interfaz `INutritionStrategy` y las clases `StandardNutritionStrategy`, `KetoNutritionStrategy` y `VegetarianNutritionStrategy`, apoyadas por la clase base `GoalBasedNutritionStrategyBase`. Con ello, `NutritionInfoController` selecciona la estrategia correspondiente según el tipo de dieta del usuario y delega el cálculo de porcentajes de macronutrientes, reduciendo duplicación de lógica y mejorando la mantenibilidad.

- **Pantallas especializadas en lugar de una vista única:**

Cada módulo del sistema se resolvió mediante una ventana específica, en lugar de concentrar todo en una sola interfaz. Esta decisión se tomó para que la navegación fuera más clara, el flujo de uso más entendible y la interfaz más organizada. También ayuda a respetar el principio de responsabilidad única en la capa visual, ya que cada pantalla atiende una función concreta: registro, consulta, perfil, información nutricional o estadísticas. Desde la perspectiva del usuario, esto hace que el sistema resulte más intuitivo y menos recargado.

- **Validaciones básicas antes de guardar:**

En los módulos de usuarios, alimentos y menús se incorporaron validaciones orientadas a prevenir registros vacíos, datos inválidos y duplicados evidentes. Esta decisión se tomó para proteger la consistencia de la información y evitar que el sistema almacene datos incorrectos desde el inicio. Aunque se trata de validaciones básicas, cumplen una función importante dentro de la calidad general del proyecto, ya que mejoran la confiabilidad de los registros y reducen errores de uso.

- **Enfoque práctico en la primera iteración:**

Considerando que esta fase del proyecto se centra en funcionalidad, estructura y cumplimiento técnico más que en una interfaz altamente estilizada, se priorizó la claridad del flujo, la correcta separación de módulos y la estabilidad de las interacciones principales. Esta decisión permitió concentrar el esfuerzo en la lógica del sistema, la persistencia de datos y la organización del código, dejando una base más firme para futuras mejoras visuales o tecnológicas.

3. Desarrollo del sistema y explicación de módulos

El desarrollo de Vitta se realizó siguiendo una estructura modular, con el fin de que cada parte del sistema atendiera una responsabilidad específica y pudiera integrarse de manera ordenada con las demás. A nivel general, la aplicación fue construida para permitir que un usuario se registre, inicie sesión, gestione alimentos, construya menús diarios, consulte información nutricional y visualice estadísticas por fecha.

La implementación se apoyó en la separación entre modelo, control y vista. De esta forma, las entidades representan la información base del sistema, los controladores procesan validaciones, cálculos y operaciones sobre los archivos, y los formularios se encargan de la interacción con el usuario. Gracias a esta distribución, el proyecto mantiene una lógica de trabajo más clara y facilita la comprensión del funcionamiento general de cada módulo.

3.1 Arquitectura, patrón aplicado y estructura del proyecto

La solución Vitta fue construida siguiendo la arquitectura **Modelo-Vista-Controlador (MVC)**, la cual permite separar la representación de los datos, la lógica de control y la interacción con el usuario. Esta decisión se encuentra alineada con lo solicitado en el proyecto, ya que la aplicación debía utilizar Windows Forms en la capa visual y mantener una estructura organizada que facilitara futuras modificaciones.

Dentro de esta arquitectura, el **modelo** representa las entidades principales del sistema, la **vista** corresponde a los formularios con los que interactúa el usuario, y la **capa de control** se encarga de validar acciones, coordinar la lógica del negocio y gestionar el acceso a los datos. Gracias a esta separación, el proyecto resulta más comprensible, más mantenible y sencillo de revisar desde el punto de vista académico y técnico.

Como patrón complementario, el proyecto utiliza Strategy en el módulo de información nutricional. La interfaz `INutritionStrategy` define el contrato del cálculo nutricional y las estrategias `StandardNutritionStrategy`, `KetoNutritionStrategy` y `VegetarianNutritionStrategy` encapsulan las reglas de distribución de macronutrientes. Además, `GoalBasedNutritionStrategyBase` concentra la lógica compartida por objetivo, mientras que `NutritionInfoController` resuelve la estrategia adecuada según la dieta del usuario.

Modelo	Controlador	Vista
Clases como User, Food y Menu, encargadas de representar la información base del sistema mediante propiedades, constructores y estructuras de datos.	Clases como LoginController, UserController, FoodController, MenuController, NutritionInfoController y StatisticsController, responsables de coordinar validaciones, cálculos, consultas y acceso a la persistencia. En información nutricional, se apoyan además en INutritionStrategy y las estrategias StandardNutritionStrategy, KetoNutritionStrategy y VegetarianNutritionStrategy.	Formularios LoginView, RegisterView, DashboardView, ProfileView, FoodRegisterView, FoodConsultView, MenuRegisterView, NutritionInfoView y StatisticsView. Capturan acciones del usuario y muestran resultados.

3.1.1 Organización del repositorio y estructura física del proyecto.

El repositorio de Vitta fue organizado para mantener separadas la documentación, los datos y el código fuente del sistema. En la raíz del proyecto se encuentran archivos principales como Vitta.sln, README.md, la carpeta docs, la carpeta data y la carpeta src.

La carpeta docs guarda los documentos de apoyo y entrega del proyecto, como el entregable inicial, el archivo con los enlaces solicitados y la documentación final. La carpeta data contiene users.csv, foods.csv y menus.csv. La carpeta src contiene el código fuente de la aplicación y dentro de ella se agrupan los tres proyectos principales que forman la arquitectura MVC.

Dentro de src, la carpeta VittaModel contiene las entidades principales del sistema, como usuarios, alimentos y menús. La carpeta VittaController reúne la lógica de control, los controladores del sistema y también las subcarpetas Abstractions, donde están las interfaces y contratos, y Strategies, donde se implementa el patrón Strategy para el cálculo nutricional. Por otro lado, la carpeta VittaView contiene los formularios de Windows Forms que conforman la interfaz gráfica, además de la subcarpeta Configuration, donde se centralizan elementos de configuración compartidos.

Esta forma de organizar el proyecto permite ubicar más fácilmente cada parte del sistema y entender mejor qué hace cada capa. También ayuda a que el código se vea más ordenado, más claro y fácil de revisar.

Además, esta estructura fue importante porque permitió trabajar el proyecto de una forma más organizada desde el inicio. Al tener separadas las carpetas y responsabilidades, fue más sencillo identificar dónde iba cada archivo, hacer cambios sin enredar otras partes del sistema y mantener una lógica más clara durante el desarrollo. Esto también deja una mejor base para seguir ampliando el proyecto después.

3.2 Pantallas y módulos principales

Pantalla / módulo	Descripción funcional	Capa principal
LoginView	Permite al usuario autenticarse y redirigirse al Dashboard. También habilita el acceso a la pantalla de registro.	Vista
RegisterView	Captura los datos del nuevo usuario, incluyendo objetivo, nivel de actividad, tipo de dieta, edad y sexo.	Vista
DashboardView	Funciona como menú principal y punto central de navegación a todos los módulos.	Vista
ProfileView	Muestra la información del usuario actual y permite actualizar sus datos.	Vista
FoodRegisterView	Permite agregar nuevos alimentos al catálogo nutricional.	Vista
FoodConsultView	Muestra la lista de alimentos, habilita búsqueda, edición y eliminación.	Vista
MenuRegisterView	Permite armar el menú diario por tiempos de comida, cargar menús previos, actualizarlos o eliminarlos.	Vista
NutritionInfoView	Resume calorías de mantenimiento, IMC, clasificación del IMC y distribución sugerida de macronutrientes.	Vista
StatisticsView	Consulta consumo diario o por período, calcula metas, días cumplidos y diferencias entre meta y consumo.	Vista

3.3 Controladores principales y su responsabilidad

Además de las vistas, el sistema se apoya en una capa de control que coordina la lógica de funcionamiento de cada módulo. Su papel principal es recibir la información de las pantallas, validarla, procesarla y relacionarla con los archivos de persistencia.

- **LoginController:** centraliza operaciones de inicio de sesión, registro y actualización delegada al controlador de usuarios.

- **UserController:** valida usuarios, comprueba duplicados y persiste cambios en data/users.csv.
- **FoodController:** administra el catálogo nutricional, permite búsquedas por nombre y persiste cambios en data/foods.csv.
- **MenuController:** registra, consulta, actualiza y elimina menús por usuario y fecha, trabajando sobre data/menus.csv.
- **NutritionInfoController:** calcula calorías de mantenimiento, IMC y distribución de macronutrientes utilizando Strategy según el tipo de dieta.
- **StatisticsController:** interpreta el contenido de los menús y construye resúmenes diarios o por rango de fechas.

3.4 Relación general entre módulos

El funcionamiento del sistema sigue una secuencia lógica. Primero, el usuario se registra o inicia sesión; después puede actualizar su perfil, administrar el catálogo de alimentos y registrar sus menús diarios. A partir de esos datos, el sistema genera información nutricional y estadísticas por fecha.

Esta relación entre módulos permite que Vitta funcione como una solución conectada, donde la información registrada en una parte del sistema sirve de base para otras funcionalidades. Gracias a ello, el proyecto mantiene coherencia interna y logra integrar acceso, gestión, cálculo y análisis dentro de una misma aplicación.

4. Uso, ejecución e interacciones del sistema

Esta sección resume la forma en que se ejecuta y se utiliza Vitta dentro de su primera iteración. Su objetivo es mostrar, de manera práctica, qué se necesita para abrir el proyecto, cuál es el orden recomendado de uso y cuáles son las principales interacciones que puede realizar el usuario dentro del sistema. De esta manera, el manual complementa la documentación técnica con una guía breve y visual del funcionamiento general de la aplicación.

4.1 Requisitos previos

Para ejecutar correctamente el sistema se requiere lo siguiente:

Elemento	Descripción
Entorno de desarrollo	Visual Studio 2022 o una versión compatible con proyectos de Windows Forms
Solución principal	Archivo Vitta.sln
Archivos de datos	Acceso a los archivos CSV del proyecto: data/users.csv, data/foods.csv y data/menus.csv.
Configuración	La configuración ya está centralizada en ConfigurationItems.cs y los archivos de datos se copian al directorio de ejecución al compilar.

Elemento	Descripción
Dependencias	Restauración automática de paquetes al compilar el proyecto

4.2 Pasos para ejecutar el sistema

1. Abrir el archivo Vitta.sln desde Visual Studio.
2. Verificar que la carpeta data contenga users.csv, foods.csv y menus.csv.
4. Compilar la solución para restaurar dependencias, copiar los archivos de datos al directorio de ejecución y validar que no existan errores de compilación.

4.3 Flujo del sistema

El flujo de uso de Vitta sigue una secuencia simple y ordenada. El usuario inicia sesión o se registra, accede al panel principal y desde ahí navega hacia los diferentes módulos del sistema. A



continuación, se presenta el recorrido visual recomendado con base en las pantallas implementadas en el proyecto.

4.4 Interacciones importantes del sistema

Interacción	Qué realiza el sistema	Resultado visible
Registro de usuario	Valida campos obligatorios y luego guarda el usuario.	Nuevo registro persistido.
Alta de alimento	Guarda nombre y macronutrientes del alimento.	Catálogo ampliado.
Búsqueda de alimento	Filtra alimentos por nombre.	Lista reducida para consulta.
Guardado de menú	Asocia usuario, fecha y tiempos de comida con alimentos y cantidades.	Nuevo menú disponible.
Carga por fecha	Recupera el menú del usuario en una fecha específica.	Formulario rellenado.
Cálculo nutricional	Obtiene calorías, IMC y macros con base en datos del usuario.	Resumen inmediato.
Consulta estadística	Recorre los menús y suma nutrición por fecha o período.	Totales, promedios y diferencias.

4.5 Cálculos principales implementados

Calorías de mantenimiento: El sistema calcula una meta calórica diaria multiplicando el peso del usuario por un factor según su nivel de actividad. Los factores documentados en Vitta son: 30 para sedentario, 35 para ligero, 40 para moderado y 45 para alto.

Fórmula: $\text{Calorías de mantenimiento} = \text{Peso} \times \text{Factor de actividad}$

Índice de masa corporal (IMC): El IMC se obtiene dividiendo el peso entre la altura al cuadrado. Si la altura fue registrada en centímetros, el sistema primero la convierte a metros antes de aplicar la fórmula.

Fórmula: $\text{IMC} = \text{Peso} / (\text{Altura} \times \text{Altura})$

Clasificación del IMC: Una vez calculado el IMC, el sistema muestra una clasificación general según rangos como bajo peso, normal, sobrepeso u obesidad.

Distribución de macronutrientes: El sistema genera una distribución referencial de proteínas, carbohidratos y grasas según el objetivo y el tipo de dieta del usuario. Esta lógica se resuelve

mediante Strategy. En la versión actual, por ejemplo, la dieta keto utiliza 30% proteínas, 10% carbohidratos y 60% grasas; la dieta estándar para mantener utiliza 25% proteínas, 50% carbohidratos y 25% grasas; y la vegetariana para mantener utiliza 20% proteínas, 55% carbohidratos y 25% grasas.

Consumo y metas por fechas: En estadísticas, el sistema lee los alimentos registrados en data/menus.csv, identifica cada alimento y su cantidad, y luego lo relaciona con data/foods.csv para obtener calorías, proteínas, carbohidratos y grasas.

5. Análisis de resultados

5.1 Validación técnica del proyecto

La validación técnica del proyecto se realizó revisando la organización general de la solución, la distribución de responsabilidades y la coherencia entre módulos, controladores, vistas y persistencia. A partir de esta revisión, se identifican los siguientes puntos de cumplimiento:

- La solución se encuentra separada en tres proyectos principales: modelo, controlador y vista.
- Las entidades base del dominio fueron definidas de manera explícita mediante clases como User, Food y Menu.
- La lógica principal del sistema se concentra en controladores especializados, evitando cargar excesivamente los formularios.
- El acceso a archivos se apoya en una implementación reutilizable mediante FileHandler<T>.
- La interfaz se desarrolla con Windows Forms, tal como lo solicita el proyecto.
- La solución incorpora revisión de buenas prácticas mediante el paquete SonarAnalyzer.CSharp.

En conjunto, estos elementos evidencian que el proyecto no fue construido únicamente para funcionar, sino también para mantener una estructura técnica clara y coherente con lo solicitado en la primera iteración.

5.2 Validación del conjunto de datos para entrega

Para fortalecer la entrega final, se trabajó con un conjunto de datos amplio y consistente para sustentar las consultas, cálculos y estadísticas. En la versión revisada, users.csv contiene 26 usuarios en total; de ellos, 25 cuentan con historial completo de menús durante 90 días y 1 usuario adicional queda disponible para pruebas complementarias.

Criterio	Resultado	Comentario
Usuarios registrados	26	26 en users.csv; 25 con historial completo y 1 disponible para pruebas.

Alimentos disponibles	84	Supera el mínimo solicitado.
Registros de menú	2250	Suficiente para estadísticas por fecha y período.
Días por usuario	90 - 90	Los 25 usuarios con menús tienen 90 días completos.
Cobertura temporal	3 meses	Cobertura continua de 3 meses.

6. Aprendizajes del desarrollo

Uno de los principales aprendizajes del proyecto fue comprender que una aplicación pequeña también necesita organización desde el inicio. Separar entidades, controladores y vistas simplificó tanto la implementación como la corrección de errores, y evitó que la interfaz se convirtiera en el lugar donde ocurre toda la lógica.

También se reforzó la importancia de planificar antes de programar. El entregable inicial permitió descomponer el trabajo en funcionalidades, PBIs y tareas, lo que facilitó distribuir el avance y entender mejor el alcance real del sistema.

Desde el punto de vista técnico, el proyecto ayudó a practicar lectura y escritura de archivos, validaciones de entradas, trabajo con fechas, cálculo de métricas nutricionales y navegación entre ventanas de Windows Forms. Además, el uso de SonarAnalyzer hizo visible que la calidad del código no depende solo de que compile, sino también de cómo está estructurado.

7. Conclusiones

Vitta cumple con el propósito de esta primera iteración al ofrecer un sistema funcional de gestión nutricional desarrollado en C# y Windows Forms, con una estructura organizada y coherente con la arquitectura MVC solicitada.

La documentación técnica demuestra cómo se tomaron las decisiones de diseño, cómo está compuesto el proyecto, cómo se ejecuta y de qué forma interactúan entre sí sus módulos. Esto fortalece la defensa del trabajo porque el sistema no se presenta únicamente como código, sino como una solución comprensible y justificable.

Finalmente, la ampliación del conjunto de datos a tres meses por usuario mejora la calidad de la evidencia del proyecto, ya que las consultas y estadísticas dejan de depender de pocos registros aislados. Con ello, el sistema se presenta no solo como un prototipo funcional, sino como una base seria para una segunda iteración más robusta.

8. Bibliografía

Repositorio del proyecto: <https://github.com/Nat-92CR/ProyectoVitta.git>

Herramienta de seguimiento: <https://trello.com/b/3z5MMCwt/proyecto-sistema-de-gestion-nutricional-vitta>

Archivos base de la solución: Vitta.sln, data/users.csv, data/foods.csv, data/menus.csv, src/VittaModel, src/VittaController y src/VittaView.

Nota de entrega: Este documento debe guardarse dentro de la carpeta docs del repositorio y acompañarse del archivo TXT con los enlaces solicitados, tal como lo establece el proyecto. Para la revisión visual del repositorio conviene presentar una copia limpia sin carpetas temporales de compilación.